SCHOTT WFS:

Understanding the WFS (BASIC)

By: Afifi

SCHOTT WFS – Source Code Explained

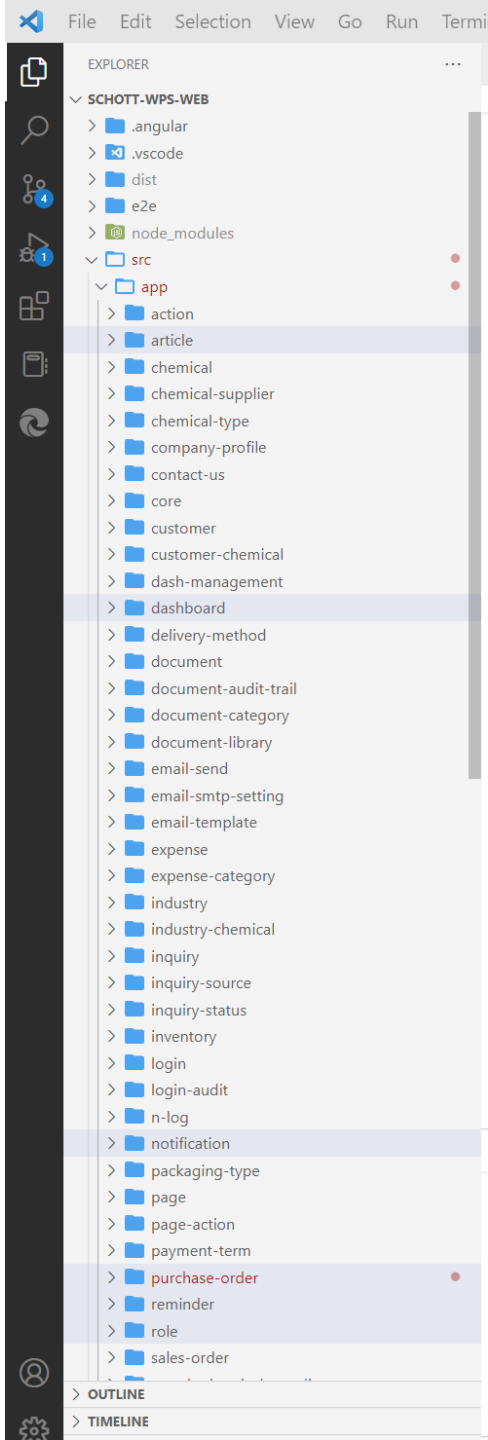
- WFS is designed based of the Angular + ASP.Net framework that come from template called Chemistry CRM.
- Chemistry CRM contained many features to support Customer Relationship Management that can be utilized for many project and further customization.
- The Database is based of Microsoft SQL Server 2019. The ASP.Net focus as Webservices (available with Swagger access).
- Angular is based v14 (Latest), with NodeJS (Latest). We used Microsoft IIS (Latest) to deploy the Production release version (minified) compilation for Angular and DLL for ASP.Net.
- There is lots of function available however we are only focus following core function for complete to cater Schott WFS architecture:
  - Articles
  - Dashboard
  - Notification
  - Reminders
  - Purchase-Order
  - Role
  - Users
- In the WFS, stand for Work Flow System, the plan to make a process flow automatically move from process to another process with easy changing the flow.

- However, we are not include with customization inside the process but focus on the priority of process set on the Articles.

- Inside the process means to cater the Form that require to fill by the PIC. This part we are hard code in order to lock customer requirement.

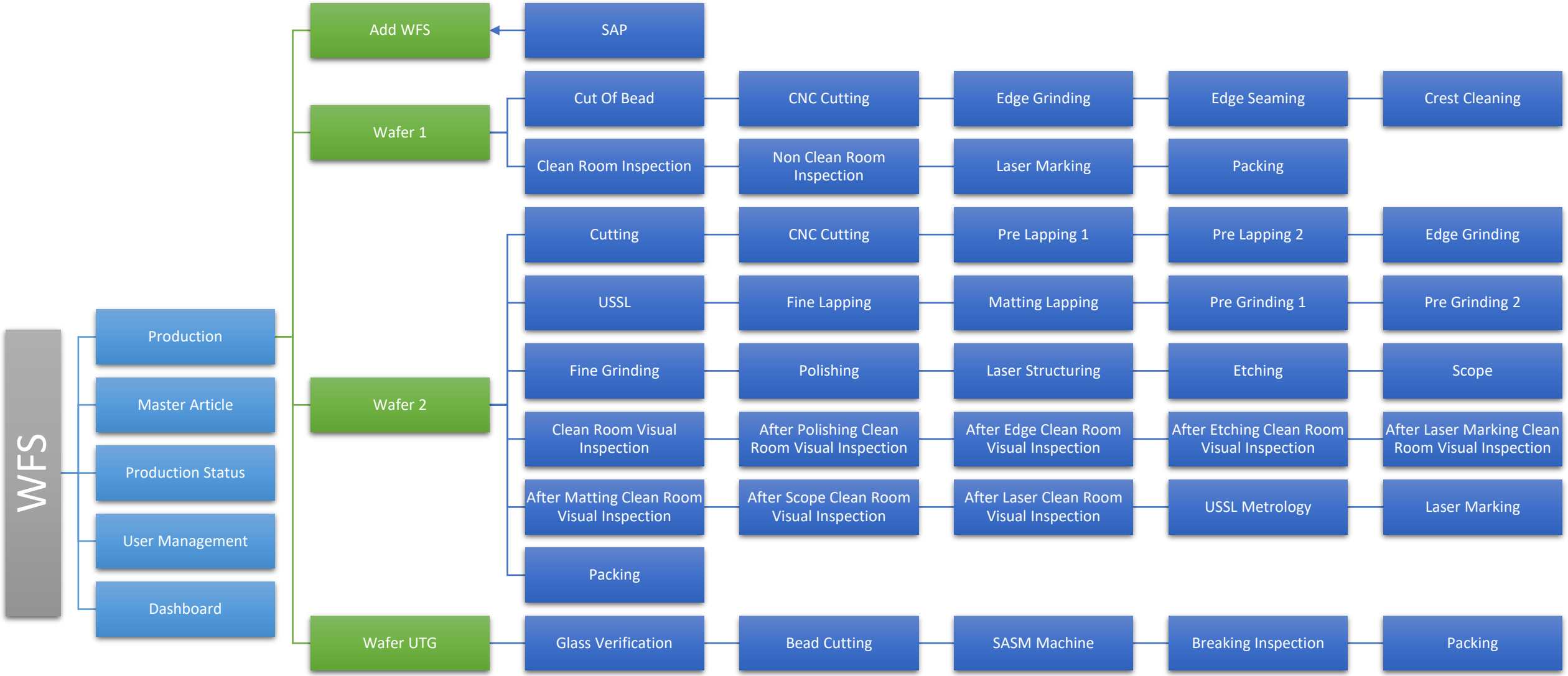
- SCHOTT have 3 main Production Flow: Wafer 1, Wafer 2 and Wafer UTG. Each of it have different number of process. Overall we have covered all the process and enable the process routing.

- This we called it v1.0, as final version and delivered to customer VM server.



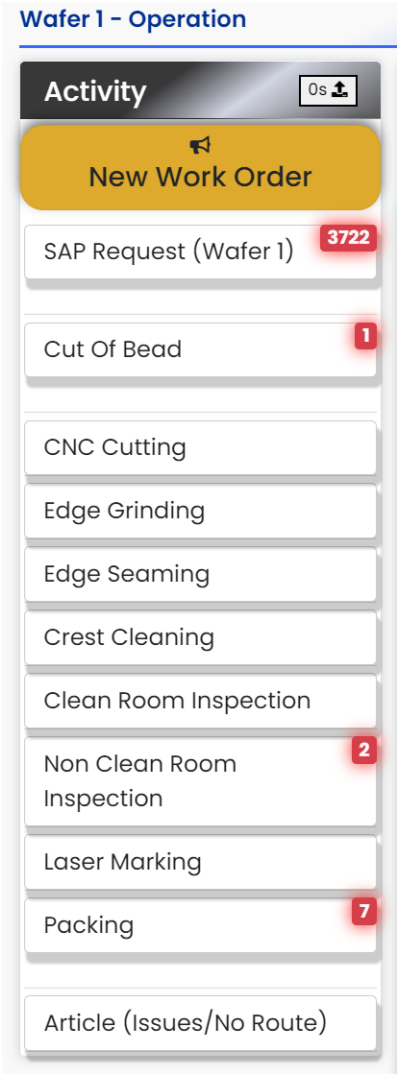
SCHOTT WFS – Source Code Explained (Con't)

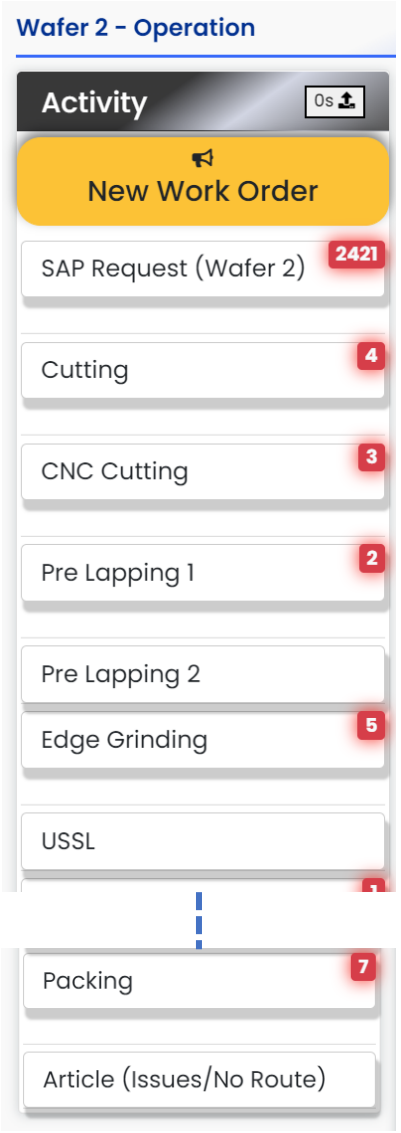
- Following are the WFS core interface:

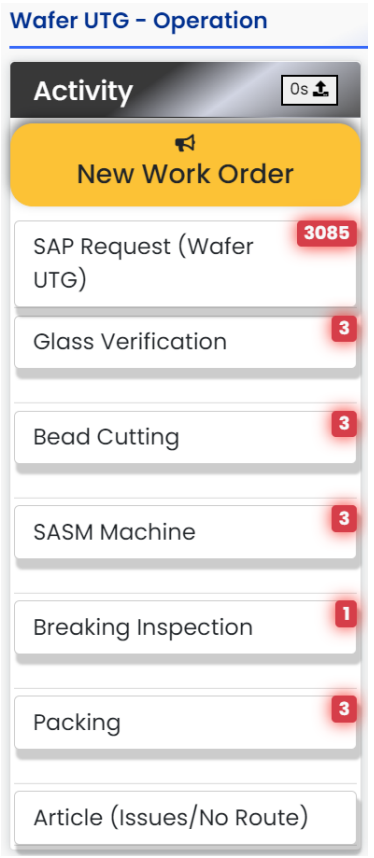


SCHOTT WFS – Source Code Explained (Con't)

- Following are the WFS core interface inside the Web:







- Article Routing is IMPORTANT, maintainer must make sure all article are provide with the routing flow contained a right CODENAME.

SCHOTT WFS – Source Code Explained (Con't)

- For Article Routing, we defined process configuration by following JSON setting that stored on form_config.json inside the App/assets, example inside:

src > assets > {...} form_config.json > [ ] form_config > {} 0 > [ ] forms

```
{
    "form_config":
    [
        {
            "id": "Wafer 1",
            "production": 0,
            "forms":[
                {
                    "id":"COB",
                    "process": 0, "table": "app_Process_W1_Cut_Of_Bead", "title": "Cut Of Bead",
                    "defect_obj": [
                        {"label": "Bubbles", "data": "bubbles"},
                        {"label": "Bubble Lines", "data": "bubble_lines"},
                        {"label": "Scratches", "data": "scratches"},
                        {"label": "Cullets", "data": "cullets"},
                        {"label": "Crack from Raw Glass", "data": "crack_rg"},
                        {"label": "Crack during Cutting", "data": "crack_c"},
                        {"label": "Light Point", "data": "light_point"},
                        {"label": "Cutting Edge", "data": "cutting_edge"},
                        {"label": "Cutting Edge From Raw Glass", "data": "crack_erg"},
                        {"label": "Chip from RG", "data": "chips_erg"},
                        {"label": "Chips", "data": "chips"},
                        {"label": "Breakage during Break", "data": "stain"},
                        {"label": "Checking", "data": "checking"},
                        {"label": "Red Line", "data": "red_line"},
                        {"label": "Others", "data": "others"}
                    ],
                    "machine": ["COB 1", "COB 2", "COB 3"]
                },
                {
                    "id":"CNC",
                    "process": 1, "table": "app_Process_W1_CNC_Cutting", "title": "CNC Cutting",
                    "defect_obj": [
                        {"label": "Bubbles", "data": "bubbles"},
                        {"label": "Scratches", "data": "scratches"}
```

Below is configuration description:

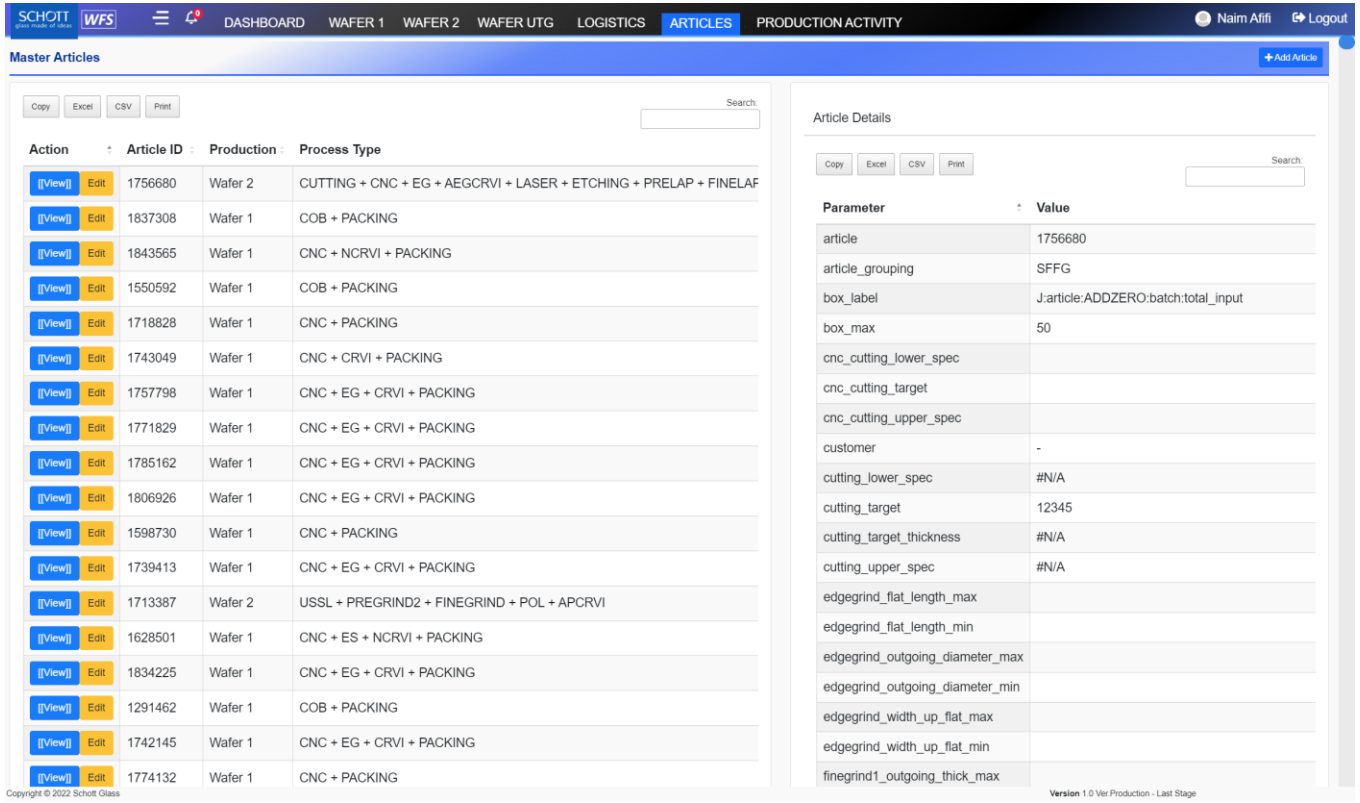
- id: "Production Name"
- Production: "Index for programming"
- Forms:
  - Id: "Codename for Process Routing"
  - Process: "Index for programming"
  - Table: MSSQL table name
  - Title: "Title to present on the page"
  - Defect_Obj: "List of defect handle component"
    - Label: "Title of Defect"
    - Data: "Name of Column on Defect"
  - Machine: "List of Machine to select on dropdown box"

SCHOTT WFS – Source Code Explained (Con't)

- For Supervisor that maintained the Article, he/she need to know the ID of Article. The ID of Article is declared on SAP by Material staff. The Article focus on providing all parameters and requirement need for production to follow. We can said that an Article is solely work for specific product.

- In order to make Article work on the WFS, Supervisor must import a list to WFS first. If not, WFS can't proceed when new Work Order coming. However WFS can show notification with a list of unknown Article for Supervisor easy to export out the list.

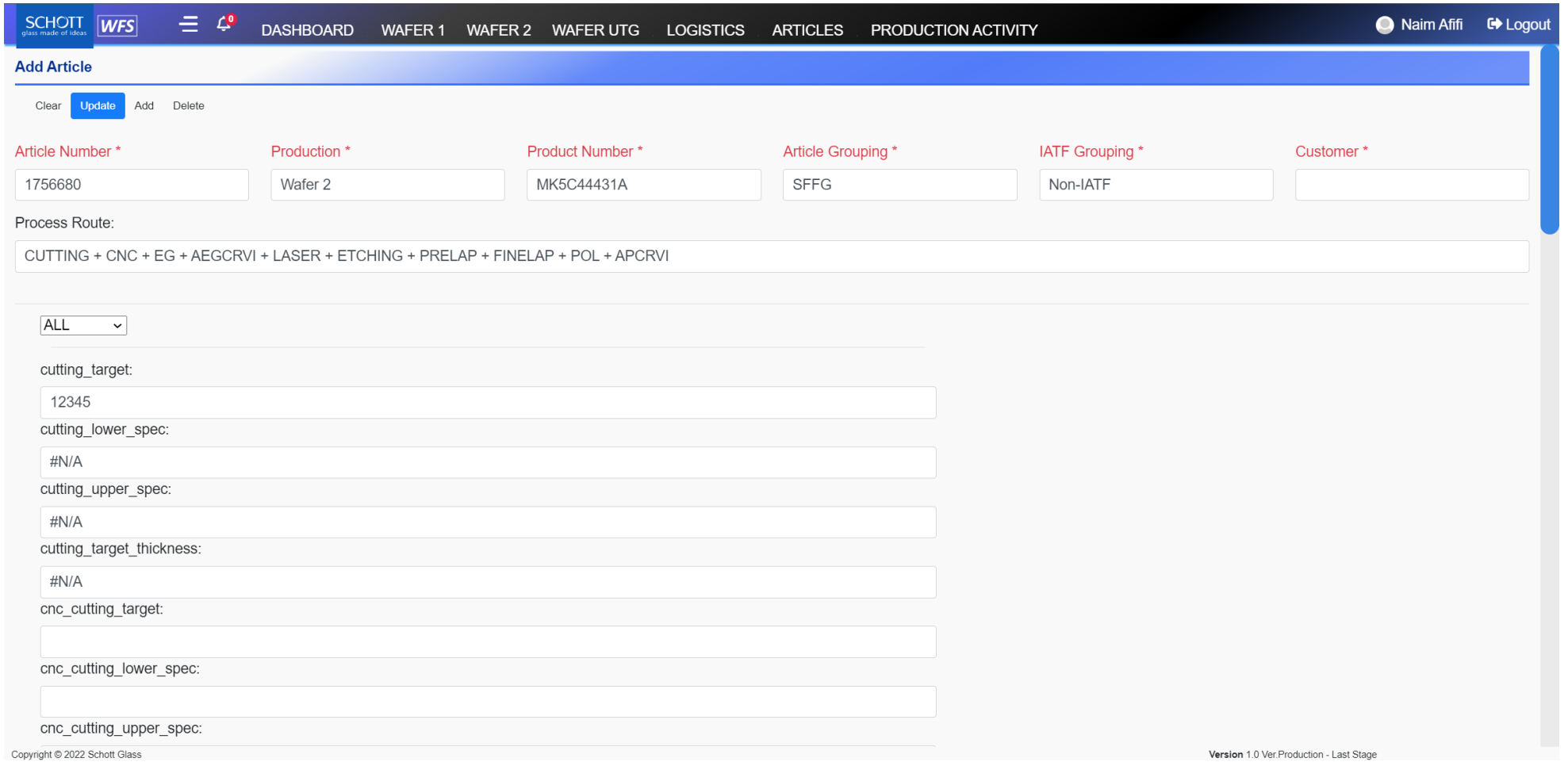
- Following are the example view of Master Article List:



- There is more then 1500 article have created by Supervisor to handle variety of customer product

SCHOTT WFS – Source Code Explained (Con't)

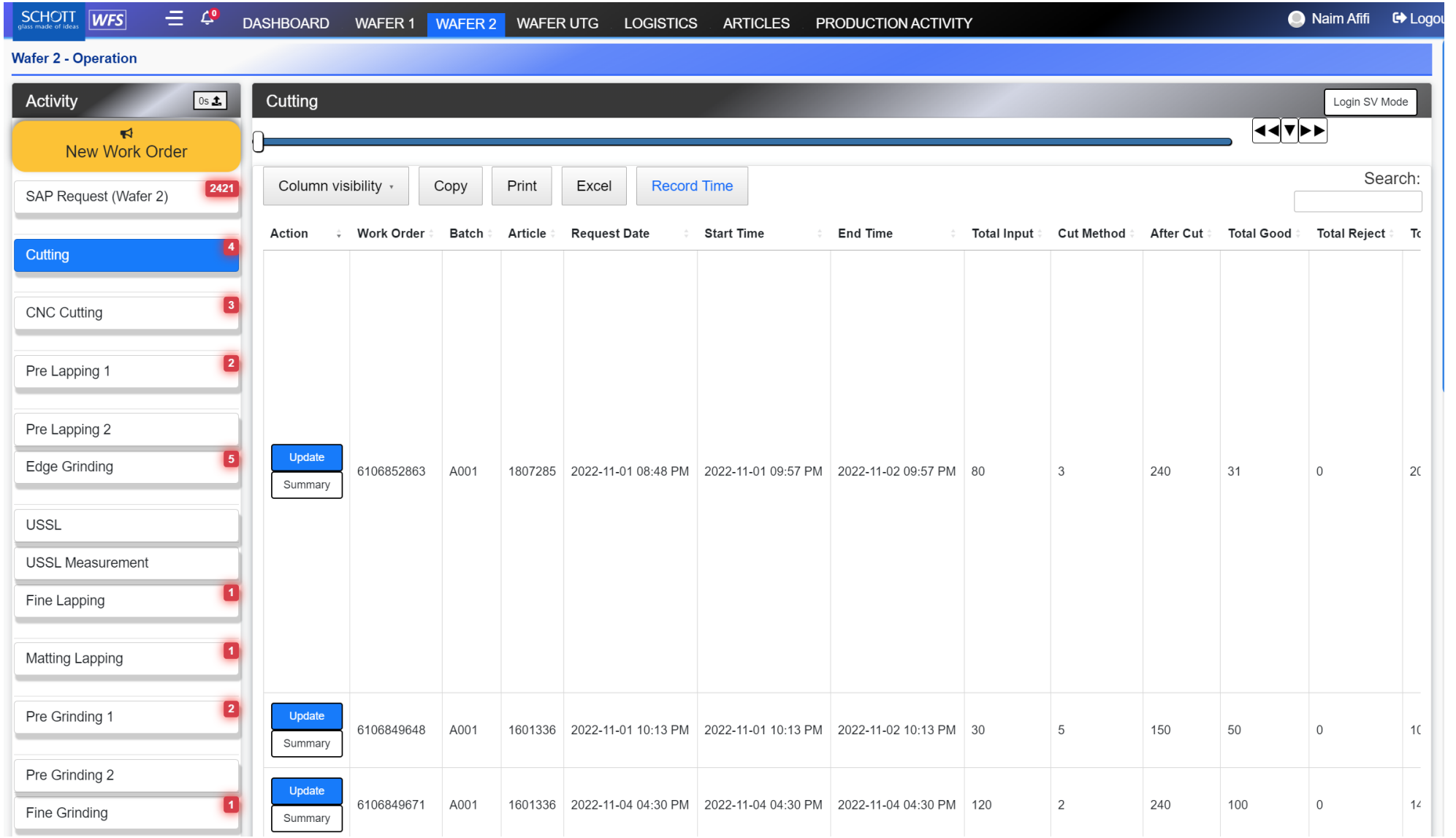
- Following each of article setting. To view this, click Edit on specific Article on Master Article.



- Each of details need to be fill otherwise the process will not work.
- Especially on Process Route. Example here show a requirement to inform WFS to provide the access for user to open the Form. After CUTTING complete it will go to CNC, and continue until APCRVI. This are code name define on form_config.json that is Fixed.

SCHOTT WFS – Source Code Explained (Con't)

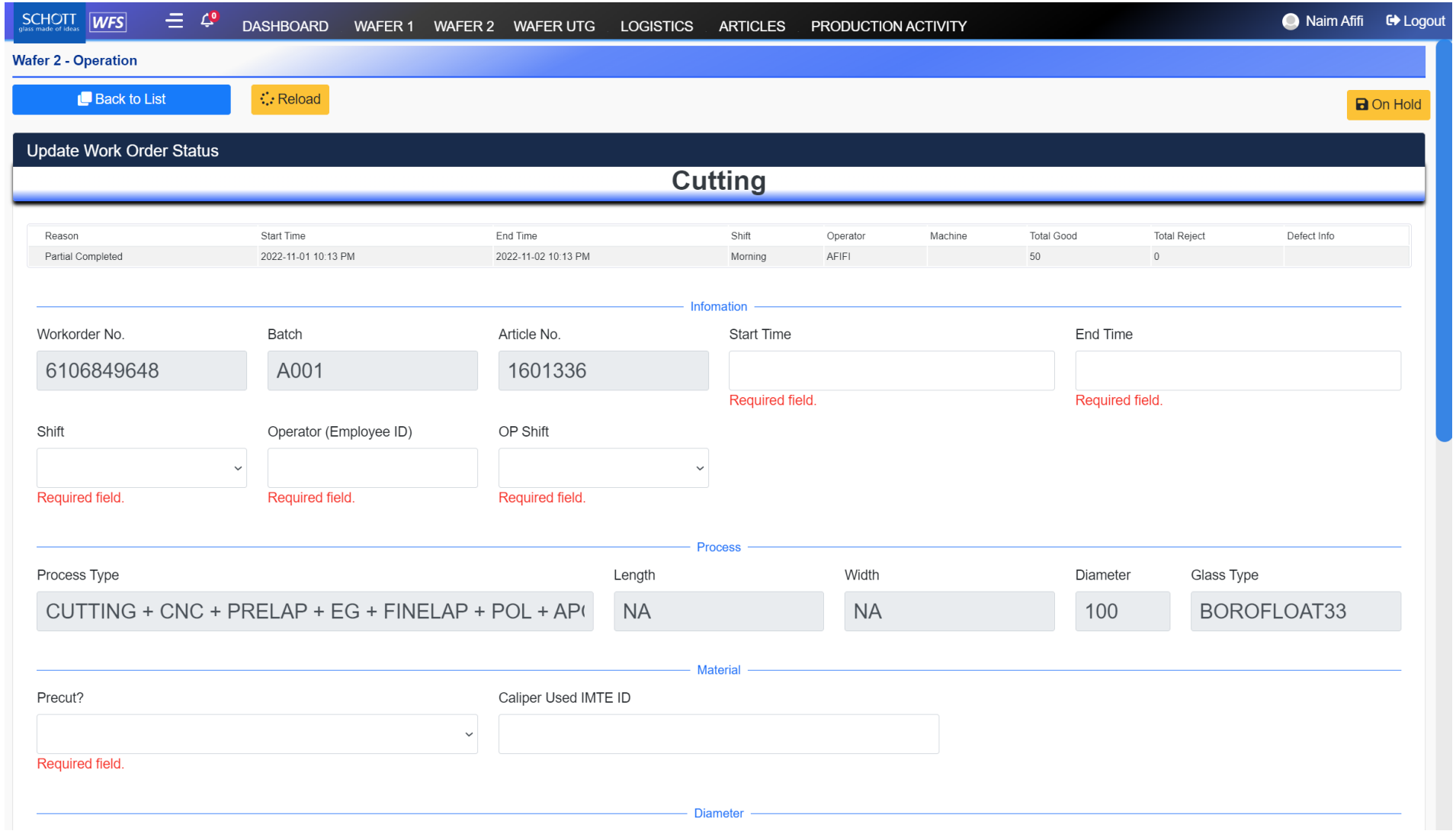
- Following are the example of when view on Production – Wafer 2 – Pending Activities



- Referring to the Activity list, there is Cutting process present 4 pending work.
- Operator can click on the process. Each of process have dedicated PIC.
- Operator can click Update to view the Form.

SCHOTT WFS – Source Code Explained (Con't)

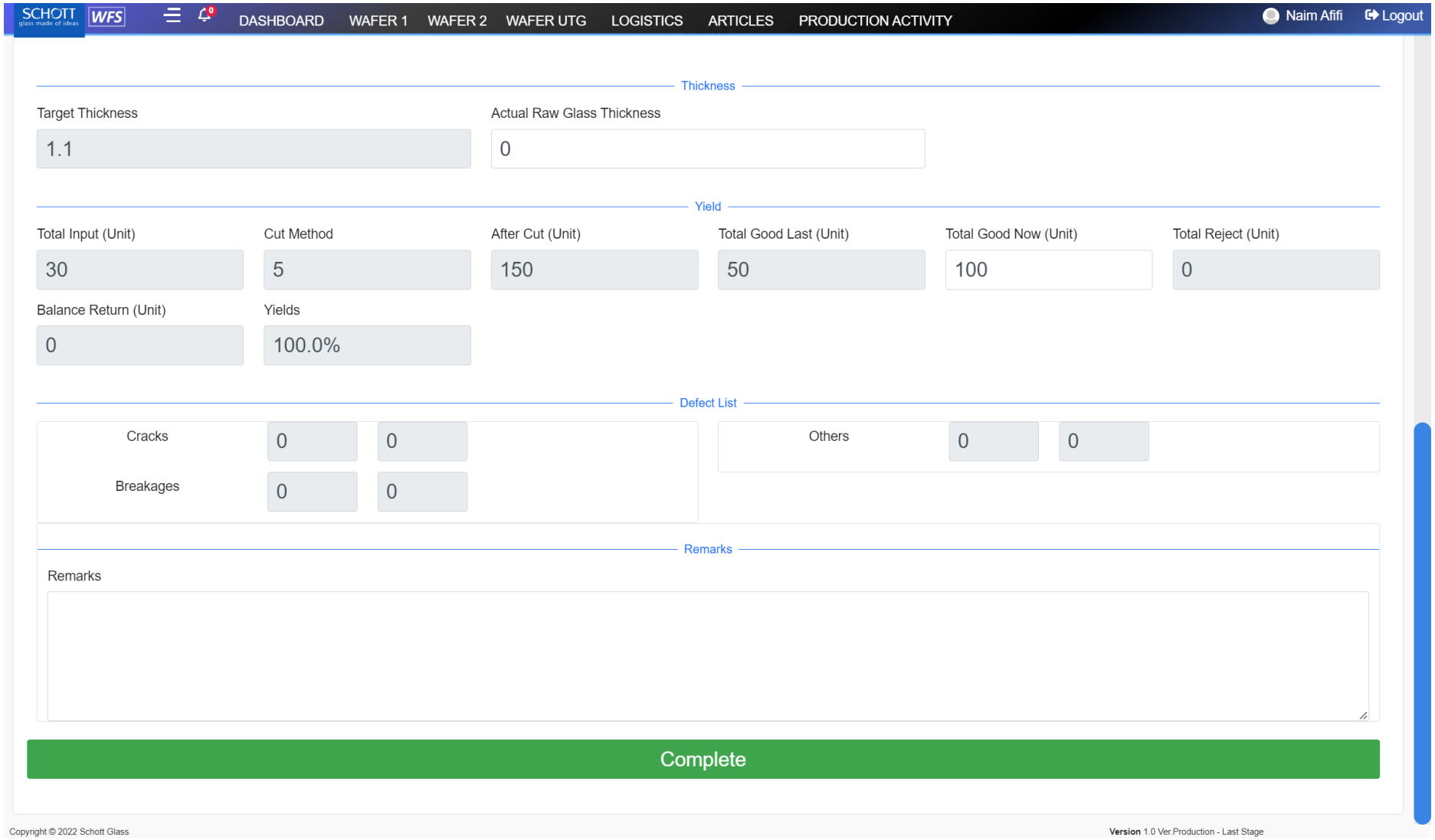
- Below example of the form on process name Cutting. Following form are create by hard code it.



- If there is issue or changes sudden at other process. User can click Reload. It will refresh.
- If there is time to wait due Supervisor or any other issue to hold, User can click On Hold. It will popup list button to record the reason to Hold.
- Back to List, it will just return to previous page safely.

SCHOTT WFS – Source Code Explained (Con't)

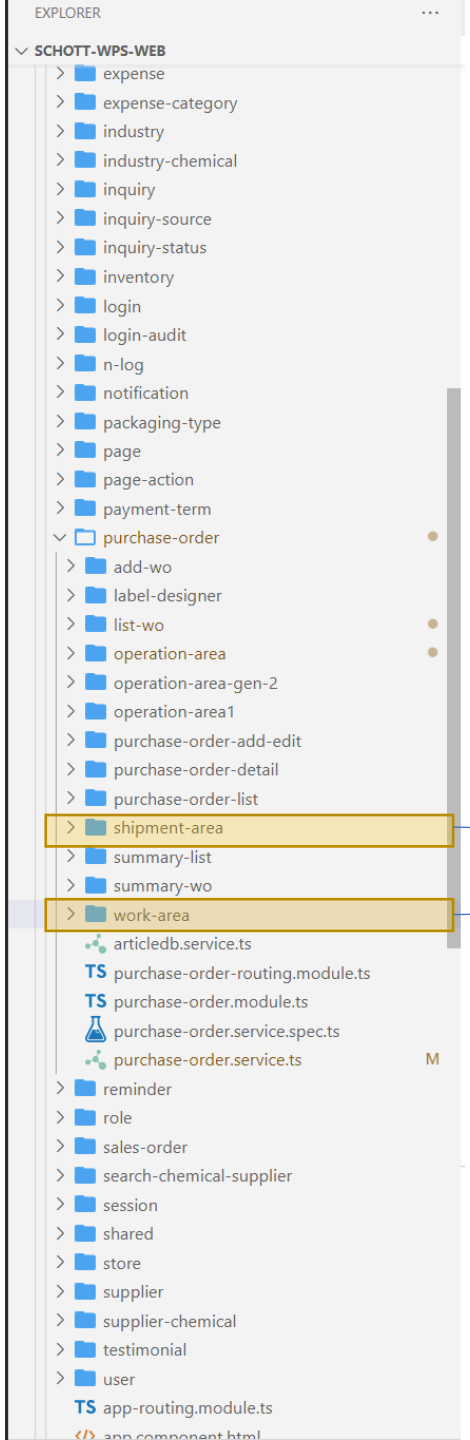
- A logic is created to cater every process. On here operator must update the quantity based of physical unit that is complete. This example part if fill complete unit. Operator can click Complete.



- However there is other condition that automatically popup when for operator to click.

- This only when the condition is met. That is:
  - Partial Complete
  - Request Topup
  - Verify
  - Rewashed

- As mentioned there is 3 main Production Flow. We need to handle differently for each of it thus, the code designed separately.



SCHOTT WFS – SHIPMENT LABEL – DEV Stage

- Based of R5 requirement provided by Schott, we will utilize WFS to handle the process flow to the Shipment Area.

- There is 2 new source folder created to handle this project.
  - Shipment-Area: To provide the Forms for Handler & QA to view/approve/print
  - Work-Area: To provide the list that cater all the Shipment routing based of R5.

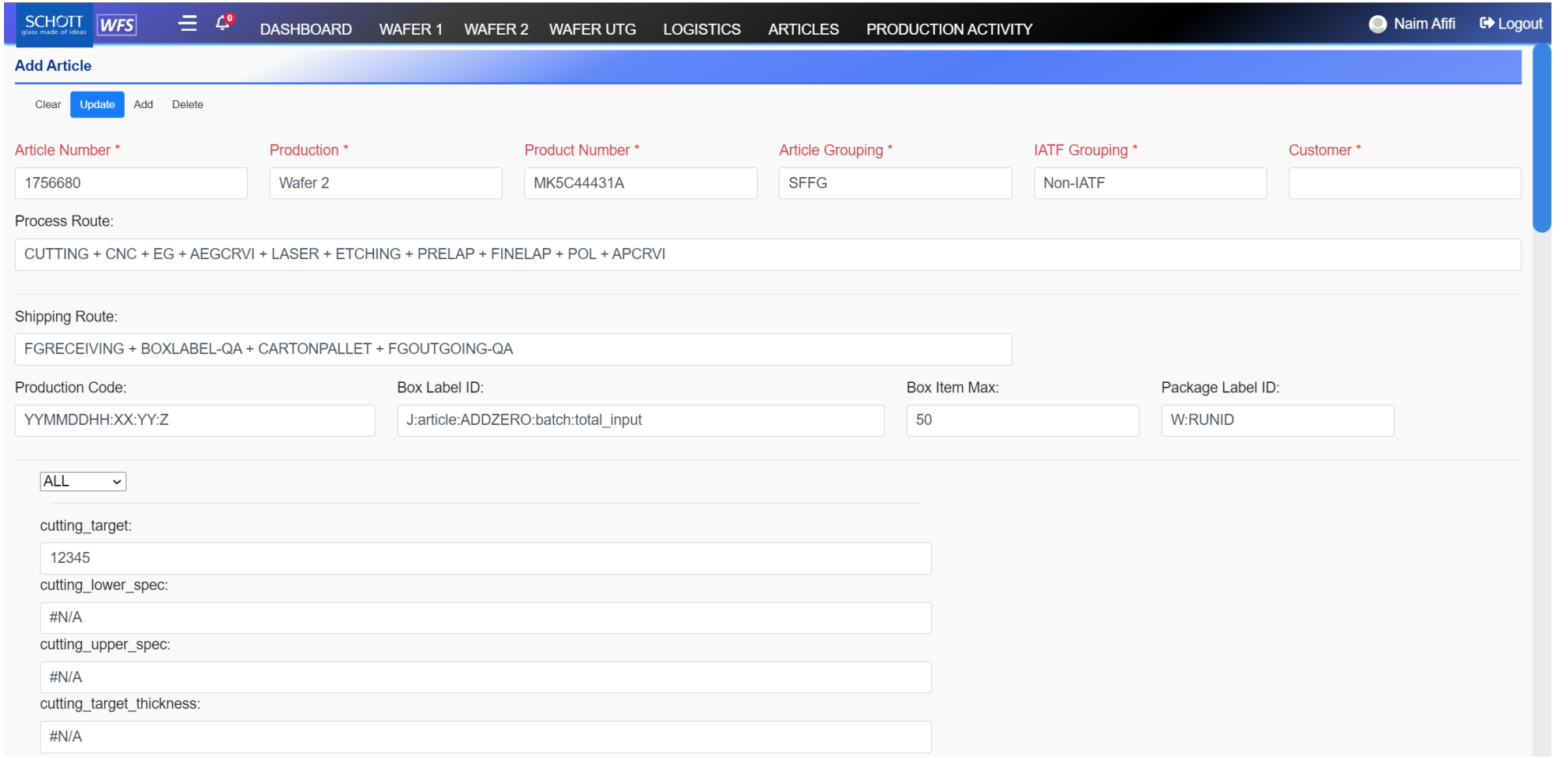
- Both source code are utilize back the existing WFS programming method.

- Please refer to the Project Planning – Shipment Label to understand the development requirement.

SCHOTT Shipment Label programming area.

SCHOTT WFS – SHIPMENT LABEL – DEV Stage

- There is additional parameters added on the Article setting. The following are yet to be finalize but we have firm the design:



- Shipping Route are similar like Process Route.
- Based on the both flow. After APCRVI end, it will move to FGRECEIVING. And the end is FGOUTGOING-QA.
- There is additional parameters to cater the Label printing requirement which automatically create ID based of the code and format, maximum to create the boxes. Thi